

[Ziteng Yang] — IC4 PhD SWE Systems Intern

Project Plan: Bitwise Equivalence and Triton Autotuning

Key People

Role	Name(s)
Intern Manager	Nick Riasanovsky
Team Manager	Alexey Loginov
Project Peers (through midpoint)	Paul Zhang , Warren Deng
Project Peers (post midpoint)	Paul Zhang , Warren Deng
XFN — Cross Functional Partners	Elias Ellison , Jason Ansel
Intern Director	William Yang

Team Summary

The **AI Compiler Performance** team is focused on helping model owners and AI kernel authors achieve SOTA performance on GPU by the simplest means possible. As a team, we primarily work on Triton and Triton Low-Level Extension (TLX) and we integrate downstream with customers using torch.compile or Helion. While one major goal is performance, correctness and accuracy are hugely important as we do not have control over our customers' downstream needs. This means an important criteria is providing high performance across various accuracy requirements. This team also functions as part of a larger V-Team generally focused on AI compiler development.

Key infrastructure we own (relevant to this project):

The majority of our team's work is maintained in <https://github.com/facebookexperimental/triton>. Typically this work is then mirrored inside fbsource/third-party/triton/beta, which we refer to as "inside fbcode". This is a fork of the upstream Triton repo.

- Triton versioning + development at Meta.
- TLX language development.
- Debugging tooling for working with Triton.

In addition, we do not directly own Triton usage inside PyTorch, but we collaborate heavily with the PyTorch team and often own subsets.

Key XFN Partners for this Project:

- PyTorch compile team

Key terms and acronyms:

- Tile-Based DSL: A GPU DSL where you program across a tile of elements (often executed by at minimum a single warp or wave). This is in contrast with languages/frameworks that have you program at the thread level (e.g. CUDA SIMT) or at the tensor level (e.g. PyTorch).
- Triton: A tile based programming language created by engineers currently working at OAI.
- TLX: Our extension to Triton that provides low-level primitives to achieve SOL. For example, two example use cases for TLX are explicit warp-specialization, the process of giving unique tasks to each warp/warp-group in a kernel, and explicit buffer reuse/sharing.
- fbcode: Code that exists inside the Meta monorepo and should be built/run using BUCK.
- fbexperimental_triton: Our fork of Triton for Triton/TLX changes found [here](#). All of these changes will be reflected inside the fbcode monorepo after landing in triton/beta.
- Bitwise Equivalence: Two kernels are bitwise equivalent if for the exact same input they produce the exact same output, down to every bit. This is typically hard to achieve because floating point math is not associative, so we typically need to enforce “floating point math is done in the same order”.
- Autotuning: The process of selecting a kernel’s parameters on a particular shape/input through benchmarking and selecting the fastest kernel. This can either be online, in which case it must be done in production, or offline, where we run autotuning separately and update a series of fixed heuristics. Sometimes a hybrid is done where a broad set of configs are tuned offline and final selection from a small set is done online.
- Layout: The orientation of data elements across the components of a GPU. These are used extensively by Triton and have a separate representation for how data is oriented in registers vs memory.
- GEMM: Generalized Matrix Multiplication
- PTX: Nvidia’s virtual machine ISA for its GPUs.
- NCU: Nsight Compute. Nvidia’s main profiler for GPU performance.

Project Summary

The Opportunity

In many DSLs like Triton, optimizing performance across a variety of shapes often requires retuning your kernel to optimize for shape specific characteristics. This is often expressed through **autotuning**, benchmarking several kernel variants and selecting the fastest one. However, Autotuning poses a challenge for correctness as these results may not be exact and we cannot directly test correctness. Here are a couple common problems:

- **Bitwise Equivalence:** Some workloads require exact results in all cases (RL, LLM training, etc). Naive autotuning cannot guarantee this.
- **Known Bugs:** Our team occasionally identifies code patterns that produce low-level bugs. Pruning all bad configs can be challenging because we may only identify the issue when it arises in the best config and disabling a config that is now working can lead to a regression.
- **Feature Benchmarking/Specialization:** Especially within development we may want to judge the performance of a specific feature or IR pattern for profiling and continued optimization.

Our goal is to expand the Triton/TLX autotuner to become aware of constraints. This should allow us to both enforce correctness and enable additional optimization given those constraints.

What You Will Build

In this project you will be focusing on bitwise equivalence, with the goal of enabling Triton's Autotuner to both achieve bitwise equivalence and high performance. To do this we will focus on teaching the Autotuner to prune "bad configs", teaching the compiler how to optimize its code generation based on specified constraints, and identifying the problem of bitwise equivalence.

You will:

1. **[Starter Task] General Purpose Autotuning Pruning** — In this milestone you will focus on generic support for pruning configs based on patterns or analysis of a compilation artifact. You will use this to filter known bugs as an initial data point.
2. **[Milestone 1] Bitwise Equivalence Reduction Detection/Enforcement** — You will construct IR/PTX for bitwise equivalence across IR/PTX. When integrated with milestone 1 we want to be able to ensure bitwise exactness for reductions.
3. **[Milestone 2] Reduction Layout Optimization** — We have added support in Triton for bitwise-equivalent reductions, but these are simply enforced for a compiler-decided layout. We would like to update the compiler to leverage the reduction decision to try and pick a "best layout" given the constraint.
4. **[Milestone 3] Bitwise Equivalence GEMM Detection/Enforcement** — As an extension of milestone 1 you will extend this analysis to the GEMM use case and enforce bitwise lowering for both Triton and TLX.

5. **[Stretch 1] Tensor Core Instruction Optimization** — We want to extend the prior work to the GEMM use case. This should ideally reduce the number of configs that need to be rejected to allow for optimizing code generation patterns (e.g. data partitioning) based on this constraint. Our goal would be to get a Triton GEMM to match CuBlas in many cases.
6. **[Stretch 2] PyTorch compiler analysis/integration** – With leftover time we can scan torch.compiler for additional kernels to enforce and extend our constraints.
7. **[Stretch 3] AMD Analysis Integration** – Fundamentally all of the work being done for Nvidia can be expanded to AMD as well. This would involve repurposing PTX based decisions for AMDGCN.
8. **[Stretch 4] GEMM + Layernorm Fusion** – You will look to optimize a GEMM + layernorm fusion kernel when programmed to maintain bitwise equivalence.

Compiler and GPU Focus Areas

This project exercises the following **GPU and Compiler engineering skills**:

- **Floating Point in ML/AI**: ML/AI workloads heavily leverage floating point in its core computation, often trying to lower the precision as much as possible to maximize computational throughput and minimize model sizes. This work is dependent on finding stability in numerics, which this project will directly reflect.
- **GPU Programming Model**: This project leverages key areas of GPU performance such as the memory hierarchy and layouts. Generating bitwise correct results requires engaging with these components directly to maintain both accuracy and performance.
- **Performance Optimization**: Several milestones will involve explicitly optimizing for performance on GPUs. In addition, in your tooling and compiler pass designs you should be aware of overhead and optimizing tooling performance accordingly.
- **GPU Profiling**: Determining optimal performance will require extensive profiling with NCU.
- **MLIR Integration**: You will expand the Triton compiler by building and extending MLIR passes. You will design your work in a way that should be abstractly visible to other MLIR passes and contains the necessary constraints.

How LLM Components Fit In

You should be heavily leveraging AI to influence the decision making and optimization in this process. In particular **you should be leveraging LLMs for verifying bitwise equivalence**, either in tooling generation, tooling analysis, or both. Most likely online LLM analysis may be too slow as the default, but you are encouraged to explore offline LLM usage or an online “developer mode” for situations in which the problem appears to be ambiguous.

AI requires extra caution for performance optimization. In particular, since the bitwise equivalence constraint is foreign to the AI, it may try to optimize performance but undoing such a requirement. You should build detailed testing frameworks to defend against such issues.

Impact and What Success Looks Like

- **An extensible autotuning framework** as a starting point for future feature development efforts across all Triton backend targets.
- **Comprehensive tooling** to analyze and bitwise equivalence across implementation. This should be extensible to future hardware/kernel specifications.
- **Reduced latency** for “bad reductions configs” by 50% relative to the “unordered” option. Outstanding work would improve the performance of the “best” config by 5-10%.
- **First ever support for bitwise equivalent Triton GEMM with Cublas.**

AI Tool Usage

AI tools are expected throughout. Specifically for this project you will want to leverage claude code, codex, or agentic frameworks for at least the following tasks:

- Correctness/bitwise equivalence analysis.
- Experiment generation/verification
- Performance investigation and hypothesis verification.
- Any mechanical components associated with LLVM/MLIR compiler generation.

A common workflow that you should expect is to have a repeatable pattern that you first orchestrate through AI and then could benefit from having the AI develop tooling for standardizing the experience.

Engineering Basic Training

You should complete the standard **virtual self-paced engineering training** covering Source Control, IDE setup, Developer Environment, and First Webpage.

Specific direction: After basic training, prioritize setting up your OSS environment for running Triton as well as PyTorch. You will be provided access to multiple different Nvidia GPUs.

Please reach out to [Nick Riasanovsky](#) if you get blocked during training.

Ramp-up Resources

- Triton Wiki: https://www.internalfb.com/wiki/Triton_Core
 - Search through all of the relevant subpages.
- Bitwise Equivalence Posts:
 - <https://fb.workplace.com/groups/420659799592399/permalink/1245881820403522/>
 - <https://fb.workplace.com/groups/257735836456307/permalink/1101398988756650/>

- <https://fb.workplace.com/groups/257735836456307/permalink/1097171219179427/>
- <https://fb.workplace.com/groups/257735836456307/permalink/1008914801338403/>
- Triton Learnings:
 - https://www.internalfb.com/wiki/Triton_Core/TorchInductor_Performance_Benchmark
- Relevant posts of induction fusions (motivations/context):
 - <https://fb.workplace.com/groups/420659799592399/permalink/1073361240988915/>
 - <https://fb.workplace.com/groups/420659799592399/permalink/1219301593061545/>
 - <https://fb.workplace.com/groups/257735836456307/permalink/973627601533790/>
 - <https://fb.workplace.com/groups/257735836456307/permalink/1061718212724728/>

Milestone Tasks and Timeline

Starter Tasks (Weeks 1-2): General Purpose Autotune Pruning

Your initial start tasks are a mix of a ramp up onto Triton itself as well as a deeper understanding of the Autotuner's process.

Task 1 (Trivial — Day 1): Nvidia Collaboration Day Bitwise Equivalence

- Attend our Nvidia collaboration in MPK and take notes on the bitwise equivalence related topics.
- **Goal:** Understand how to inspect the compiler, the general IR changes through the compiler pipeline, and the stage level artifacts (e.g. `make_ttgir`).

Task 2 (Trivial — Day 2-3): Understand the Compilation Pipeline and Autotuning Flow

- Generate 10-15 (AI assisted) Triton/TLX kernels that demonstrate the transformations produced by various compiler stages.
- Analyze the code base for documentation gaps and update docs (with AI) for any under-documented passes that can modify numerics.
- **Goal:** Understand how the compiler transformations change the IR and what can be leveraged for Autotuning. Please aim to land these as a new set of examples/tutorials for onboarding users.

Task 3 (Moderate — Week 1-2): Integrate Support for correctness checks in Autotuning

- Build a hook into the Autotuner that can “test” the autotuning result on some predefined output with a user-defined correctness function.
- The design should align with “simplifying” our testing infrastructure and should include the ability to “inspect the success rate”.

- Apply this to the FA TMEM_LOAD accuracy issue example and several correctness tests that tune over many configs.
- **Goal:** Build a testing baseline for validating all future accuracy experiments. This can also be adopted by the team as a first debugging step.

Task 4 (Moderate — Week 2): IR based correctness pruning

- Integrate Autotuner support for pruning based on IR output rather than runtime accuracy checks.
- Build examples where you filter based on the ttgir as well as the generated PTX. The necessary targets should include:
 - TMEM_LOAD filter bug.
 - A ttgir based AutoWS example (feature selection)
 - A PTX based vectorization example (feature selection)
- **Goal:** Build autotuner support as an accuracy fallback that can be used for future milestones

Milestone 1: Bitwise Equivalence Reduction Detection/Enforcement (Weeks 3-5)

Goal: Build standalone and autotuner integrated tooling to validate bitwise equivalence when comparing various Triton kernels called on the same exact input. This should function as tooling for debugging, tooling for benchmarking, and proactively identify potential gaps in the compiler that could lead to numerical differences.

Problem statement: As AI models scale larger and larger, accuracy degradation (NE issues) becomes a larger disrupter for both cost and developer productivity. For example, if you are replacing a kernel in one of TBD's models that will change the numerics, they will likely need to run a ladder test that takes 1000s of GPUs and potentially ≥ 1 week to run. One technique that has been developed is to require bitwise exact numerics for all changes within some interval. This will guarantee that the numerics will not change and ladder tests can be skipped. Similarly, there are many other use cases where bitwise equivalence can be a desirable or necessary property (e.g. batch-invariant inference, RL).

Typically this is done by keeping model components "fixed" (e.g. enforce this exact layout) and disabling tuning, effectively taking a performance penalty for accuracy. However, we believe that in many cases we can prove/enforce accuracy, allowing for greater shape based tuning and potential performance wins.

The first step in this project is this proof/analysis step. You will build support for detection for bitwise equivalence in reduction use cases and leverage that to enforce it in Autotuning. This will likely require building a larger tooling module and will need to examine the underlying PTX.

System Components to Build:

- **Design Doc:** You should draft a design doc covering your PTX analysis tooling. Key questions that should be focused on should be scaling (both kernel and PTX complexity)

and optimizing for real uses (e.g. config pruning vs retrospective) and should consider the full suite of information that may be available to us.

- **PTX Bitwise Reduction Tooling:** Standalone tooling to analyze PTX with corresponding source code and determine the bitwise ordering.
- **Experiment Framework:** You develop a repeated experimental framework for evaluating kernels generated through Triton autotuning and torch.compiler generated configs. We should apply this to a few real model examples. This should evaluate:
 - Performance benchmarking relative to the full suite of unordered reductions
 - The pruning rate due to incompatible configs.
- **Autotuning Integration:** The bitwise reduction tooling should be callable directly from the Triton autotuning, allowing the final Autotuning results to only consider bitwise equivalent options.
- **PTX support** for common numerics across changes to kernel patterns, including:
 - Reductions on multiple tensors
 - Multi-dimensional reductions
 - FMA, sum, prod operations
 - Varying reduction blocksizes (with corresponding kernel changes)
- **Example Suite:** There should be a suite of proven bitwise equivalent examples as a tutorial, both showcasing the tooling and clear example of bitwise equivalence that would be interesting to someone new to the project.
- **Progress Post:** A post indicating the work you have completed with a couple of examples.

What success looks like:

- Demonstrate the ability to accurately determine bitwise equivalence (especially compared against experimental data) on standard PyTorch implementations.
- Provide a clear design doc showcase how this tooling can be scaled to larger and more complex use cases.
- Extend support to fusion examples from 3-5 example model generated kernels.
- Leverage Autotuning integration to benchmark performance implication in an autotuned example on both H100 and B200 and assess the performance overhead of ordered reductions.
- Strong testing of the tooling and extensions of our internal tests based on the relevant examples.
- **Exceeds:** As you demonstrate equivalence between kernels, it would be ideal to land diffs representing these programmatic transformations, both from an IR and kernel level. This conceptually represents much of the “search space” for someone looking to optimize performance with bitwise equivalence.
- **Exceeds:** Agentic testing suite integration. Directly enabling agents to leverage this to verify correctness and potentially hillclimb for performance.

This milestone must be fully completed before midpoint feedback opens (week 5).

Milestone 2: Reduction Layout Optimization (Weeks 6-8)

Goal: Optimize various Triton reduction kernel patterns to at minimum reduce the risk of suboptimal configs. In the best case this will improve the best shape, although that may not be possible for all configurations.

Problem statement: Within Triton we have added support for deterministic reductions, within reason (e.g. fixed reduction BLOCK_SIZE) changes to the autotuning configurations should result in the same final output. However, this is currently done by “enforcing the same result regardless of data layout”. When this is a necessary requirement for correctness in the worst case, it's not optimal as this layout may be optimized for the “unordered” reduction assumption, which no longer holds.

To improve performance and reduce the risk of “bad heuristics”, we want to update the compiler to generate “reduction-ordering aware heuristics”. Effectively we should tell the compiler that it will do a deterministic reduction and therefore it should select the layout that it thinks will have the most performance. There may be some cases that cannot improve performance (either due to layout or fundamental math limitations), but there should be many configurations that could improve.

System Components to Build:

- **Design Doc:** You should draft a design that explains both how to assess layout cost when considering the reduction in isolation and how the layout should be weighed against other layout constraints (e.g. tmem_store, local_store, tma_load, etc.). You should design your modifications with a focus on enabling future experimentation.
- **Layout optimization pass:** A pass that should handle refactoring layouts before LLVM lowering based on the reduction requirements.
- **Experiment Framework:** A framework that can extend kernel performance measurements across “all modified configurations” not just the winning config. This will be needed to evaluate the impact on cases with suboptimal heuristics.
- **Profiling Evaluation:** You should integrate with agents/NCU ways to interpret the impact of the changes on performance. In particular you should establish the relationship between the layout changes on the impact on important metrics such as memory throughput.

What success looks like:

- Significant performance improvements (ideally closing the gap between ordered and unordered) by as much as 50% for the worst performing configurations.
- Profiling based reporting indicating the relationship between layouts and end to end performance. This should also provide insight into required core kernel changes.
- **Exceeds:** Improve e2e reduction kernel performance by optimizing the “best” config being selected.

- **Exceeds:** Implement kernel authoring changes to improve e2e performance based on the insights of the profiling data. This could include launch grid information or code structure for example.

Milestone 3: Bitwise Equivalence GEMM Detection/Enforcement (Weeks 8-10)

Goal: Ensure bitwise equivalence across algorithmically equivalent GEMMs and enforce this via Autotuning in the compiler (as well as potentially modifying compiler passes). The desired output is to produce GEMMs bitwise equivalent to CuBlas for many shapes.

Problem statement: Beyond reduction there are many other kernels that could result in outputs that are not bitwise equivalent. A particularly important class of these are GEMM kernels (or kernels that depend on MMAs in general). While there are some algorithmic results that can lead to differences (e.g. Split-K) overall the biggest change should be the underlying MMA instructions and how they influence the final output. The exact scope of this milestone will be dependent on the output of our meeting with Nvidia at the start of the internship.

In addition, for simple lowering cases, similar to the inner tree reduction, we would like to allow as many configs as possible to successfully lower to bitwise equivalent results. For example, if the requirement were $N=64$ in the MMA instruction, then we should attempt to make $BLOCK_N=128$ compatible as well by lowering to multiple $N=64$ instructions. The eventual deployment goal would be to support GEMM fusions in PyTorch, so ideally we would be able to demonstrate both GEMMs and corresponding fusions and if possible, maintain bitwise equivalence between the fused and unfused versions.

System Components to Build:

- **Design Doc:** You should construct a design focusing on how you will track the relevant MMA information both in Triton and in external tooling, what knobs are available for manipulation, and how this can be extended to future passes. You should focus on how passes can verify the legality of certain optimizations as well.
- **PTX Bitwise Reduction Tooling**
 - Extensions into GEMM specific instruction (e.g. TC ops, wgmma). This should work with H100 and B200 GPUs.
 - Extension into common Inductor generated GEMM fusions.
- **Compiler Lowering Changes:** For many common cases we should transform the lowering to support generating bitwise equivalent MMA instructions. This should be compatible with H100 and B200 GPUs and should allow for potentially larger block sizes.
- **Constraint Representation:** You will need to build an efficient representation of all MMA restrictions so they persist across compiler passes and can be used for future optimizations.
- **Performance Evaluation:** You should do extensive profiling on the impact of the numerics risks on the possible best kernel performance as well as per config performance. You should reuse the tooling from milestone 2 to evaluate the change in

performance from configs that would otherwise be pruned and the impact on core metrics like TC throughput.

- **Progress Post:** A post indicating the work you have completed with a couple of examples.

What success looks like:

- Support for Autotuning GEMM (with a manually imposed restriction) and producing bitwise equivalent results.
- Bitwise equivalent results to CuBlas across 5 example shapes. This may require experimentation to determine the CuBlas ordering.
- Benchmarking and reporting on the potential overhead of bitwise equivalent results, including mapping the impact of restriction instruction on key performance indicators.
- **Exceeds:** Experimentally calculate (and produce a repeatable process) for classifying the bitwise equivalent restrictions across the spectrum of CuBlas GEMM shapes.

Stretch Goals

Note: The stretch goal ordering is highly independent. You do not need to complete the stretch goals in the prescribed order.

Milestone 4 [Stretch 1]: Tensor Core Instruction Optimization

Goal: Similar to Milestone 2, we want to optimize Triton configs whose lowering had to change to match bitwise equivalence. Rather than focusing on layouts, this stretch goal will focus on evaluating MMA ordering, kernel level techniques such as data partitioning, and possible changes to pipelining

Problem statement: In milestone 3 you generated support for bitwise equivalence (when possible) across GEMMs. However, this was done through a straight-forward “lowering” process, similar to the state of reductions before milestone 2. In this milestone we will look to optimize the kernel given this lowering constraint.

Unlike the prior layouts we won’t be optimizing via layout conversions but instead larger kernel tuning/optimization. This milestone will be more open-ended than the prior milestones, but some key areas to check will be:

- Data Partitioning: In particular placing a larger focus on MMA op reordering
- Num-Stages: Changes to the required K size may modify the necessary pipeline depth/ideal config, including updating when we require signal TMA buffers as empty/free. This could include modifying the TMA load, but you should ensure this makes the kernel (meaningfully different) from just lowering the block size.
- Async Ops: Right now there may be additional barriers that are otherwise unnecessary due to the prior code structure.

System Components to Build:

- **Optimization Passes:** You should modify or introduce compiler passes that will optimize the kernel based
- **Benchmarking:** You will need to expand your tooling to evaluate MMA performance.

What success looks like:

- Performance improvements on “altered” configs by 10-20% for cases where these are not the best config.
- **Exceeds:** Best config performance improves by 5-10% after the changes.

Milestone 5 [Stretch 2]: PyTorch compiler analysis/integration

Goal: Review torch.compile for more complex use cases that can be generated, especially fusions. We should verify bitwise correctness (fixing any gaps) and perform tuning to determine how to update the optimal configs.

Problem statement: The eventual downstream consumer of most implementations are likely torch.compiler users. This generally requires matching the numerics of pytorch eager, either by changing eager or compile. Given the timeline of this internship we do not want to try change pytorch eager, although we will happily accept such open source contributions in the future.

However, we do need to be able to support the PyTorch eager use cases. In particular we will focus on:

1. The outputs generated by fusion (and compare them to the results of their equivalent unfused Triton kernels).
2. Search for any kernels not well represented in milestone 1.

The end result should be verifying many kernels are bitwise equivalent and demonstrating the capability to achieve performance on fused kernels.

System Components to Build:

- **Data Collection:** You will need to build a more comprehensive data collection process for evaluating PyTorch models.
- **Compiler Improvements:** You will need to modify compiler passes based on any uncovered gaps.
- **PTX Bitwise Reduction Tooling Improvements:** You should identify PTX patterns that are not yet captured by your existing tooling.
- **Benchmarking:** You should evaluate the state of performance and success, repeating the findings from milestone 2.

What success looks like:

- You will produce a comprehensive review of PyTorch functionality identifying any major gaps that appear to still remain.
- You produce a benchmarking report indicating the bitwise equivalent results for several key customer examples. These do not need to match PyTorch Eager, although that would be an exceptional result.

Milestone 6 [Stretch 3]: AMD Analysis Integration

Goal: Modify the outputs of milestone 1 to work with AMD as well and function as a starting point for enabling bitwise equivalence with AMD.

Problem statement: While our primary use cases involve Nvidia GPUs, our team generally serves hardware heterogeneity goals and this is a priority at Meta. As a result, we want to show that our techniques translate to AMD GPUs as well. To this end we want to extend the bitwise equivalence detection from PTX to AMDGCN. We will focus on achieving parity via reduction and MFMA operations.

System Components to Build:

- **AMDGCN Bitwise Reduction Tooling:** Extend the existing PTX tooling to work with AMDGCN as well. This should ideally reuse whatever components aren't PTX specific.
- **Evaluation:** You should extend your evaluation framework to handle AMD as well.
- **Compiler Change:** Should time allow, you should extend the inner_tree logic to AMDGCN as well.

What success looks like:

- Demonstrate accurate bitwise checks on AMDGCN sum/prod reduction kernels with equivalent results after autotuning.
- Verify bitwise equivalence failure for autotuning configs that mod
- Support for autotuning configuration and rejecting all that don't match a single deterministic result
- **Exceeds:** Integrate inner_tree support to enable deterministic reduction ordering on AMD.

Milestone 7 [Stretch 4]: GEMM + Layernorm Fusion

One of our customers is interested in exploring a GEMM + layernorm fusion. We will like to assess performance while considering stable numerics. Key questions we would like to answer are:

1. Can we apply this fusion and possibly match the numerics of the unfused version? Why or why not?

2. How does the performance compare when forcing “stable” numerics to the unordered version?
3. Can we extend your layout optimization from Milestone 2 to further optimize performance? How does this compare for H100 and B200?

Timeline Summary

Week	Focus	Systems Emphasis	Milestone
1-2	Ramp-up, starter tasks	Understand Triton compiler pipeline, autotuning structure, map signal landscape	Starter Tasks
3-4	Bitwise equivalence detection tooling.	Floating point numerics, deterministic testing, PTX, GPU programming model	Milestone 1
5	Bitwise equivalence autotuner integration + generalization across kernels	Compiler structure, verification, GPU programming, torch.compile	Milestone 1 (complete before midpoint)
6-8	Reduction tuning	GPU performance optimization, layouts, compiler structure	Milestone 2
8-10	Bitwise equivalent GEMM	GPU programming model, floating point numerics, compiler Compiler structure	Milestone 3
10+	TC optimization, Pytorch integration, or AMD support	GPU performance optimization, floating point numerics, AMDGCN, GPU programming model	Milestone 4/5/6 (stretch)
11-12	Polish, experimental measurements, final presentation	Documentation, runbooks, handoff	Wrap-up

Appendix: Code Examples

These are a few relevant Triton code examples to help motivate the underlying usage. More code examples can be provided upon request.

Simple Reduction

Python

```
@triton.jit
# pyre-ignore[3]: Triton JIT kernel; no meaningful return type.
def sum_dim1_kernel(
    # pyre-ignore[2]: Pointer
    input_ptr,
    # pyre-ignore[2]: Pointer
    output_ptr,
    M: int,
    N: int,
    # pyre-ignore[2]: May specialize
    input_stride_m,
    # pyre-ignore[2]: May specialize
    input_stride_n,
    BLOCK_N: tl.constexpr,
    ROWS_PER_BLOCK: tl.constexpr,
    # pyre-ignore[9]: Triton constexpr default
    REDUCTION_ORDERING: tl.constexpr = None,
):
    """Sum a 2D tensor along dim1 (columns).

    Each program instance loads a [ROWS_PER_BLOCK, BLOCK_N] tile and
    reduces along the column axis. ROWS_PER_BLOCK groups rows along
    the parallel (grid) dimension. No loop over columns – BLOCK_N must
    be >= N.
    """
    pid = tl.program_id(0)
    row_start = pid * ROWS_PER_BLOCK

    row_offsets = (row_start + tl.arange(0, ROWS_PER_BLOCK)).to(
        tl.int64
    ) # [ROWS_PER_BLOCK]
    col_offsets = tl.arange(0, BLOCK_N).to(tl.int64) # [BLOCK_N]

    # Build 2D offset grid: [ROWS_PER_BLOCK, BLOCK_N]
    offsets = (
```

```

        row_offsets[:, None] * input_stride_m + col_offsets[None, :] *
input_stride_n
    )
    mask = (row_offsets[:, None] < M) & (col_offsets[None, :] < N)

    vals = tl.load(
        input_ptr + offsets, mask=mask, other=0.0
    ) # [ROWS_PER_BLOCK, BLOCK_N]
    sums = tl.sum(
        vals.to(tl.float32),
        axis=1,
        reduction_ordering=REDUCTION_ORDERING,
    ) # [ROWS_PER_BLOCK]

    row_mask = row_offsets < M
    tl.store(output_ptr + row_offsets, sums, mask=row_mask)

```

Note: We have added the `REDUCTION_ORDERING` to specify when we want an ordered reduction. You will be optimizing for that use case.

Persistent Reduction

```

Python
@triton.jit
# pyre-ignore[3]: Triton JIT kernel; no meaningful return type.
def sum_dim1_kernel_batched(
    # pyre-ignore[2]: Pointer
    input_ptr,
    # pyre-ignore[2]: Pointer
    output_ptr,
    M: int,
    N: int,
    # pyre-ignore[2]: May specialize
    input_stride_m,
    # pyre-ignore[2]: May specialize
    input_stride_n,
    BLOCK_N: tl.constexpr,
    ROWS_PER_BLOCK: tl.constexpr,
    # pyre-ignore[9]: Triton constexpr default
    REDUCTION_ORDERING: tl.constexpr = None,
):

```

```

"""Batched sum along dim1: loops over column chunks of size BLOCK_N.

Unlike sum_dim1_kernel which requires BLOCK_N >= N, this kernel
iterates over the columns in chunks of BLOCK_N, performing a tree
reduction within each chunk and linearly accumulating across chunks.
"""

pid = tl.program_id(0)
row_start = pid * ROWS_PER_BLOCK

row_offsets = (row_start + tl.arange(0, ROWS_PER_BLOCK)).to(tl.int64)
row_mask = row_offsets < M

acc = tl.zeros([ROWS_PER_BLOCK], dtype=tl.float32)

for col_start in range(0, N, BLOCK_N):
    col_offsets = (col_start + tl.arange(0, BLOCK_N)).to(tl.int64)
    offsets = (
        row_offsets[:, None] * input_stride_m
        + col_offsets[None, :] * input_stride_n
    )
    mask = (row_offsets[:, None] < M) & (col_offsets[None, :] < N)

    vals = tl.load(input_ptr + offsets, mask=mask, other=0.0)
    batch_sums = tl.sum(
        vals.to(tl.float32),
        axis=1,
        reduction_ordering=REDUCTION_ORDERING,
    )
    acc += batch_sums

tl.store(output_ptr + row_offsets, acc, mask=row_mask)

```

For larger reductions we will often loop across multiple blocks. In select instance the inner reduction can equal the tree reduction, depending on the order (e.g. BLOCK_N=128 tree vs BLOCK_N=64 persistent).

Welford Reduction

Python
@triton.jit

```

# pyre-ignore[3]: Triton JIT kernel; no meaningful return type.
def welford_combine(mean_a, m2_a, count_a, mean_b, m2_b, count_b):
    """
    Combine two Welford partial results using the standard
    parallel/pairwise Welford merge formula.
    """
    count = count_a + count_b
    delta = mean_b - mean_a
    new_mean = mean_a + delta * (count_b / tl.maximum(count, 1.0))
    new_m2 = m2_a + m2_b + delta * delta * count_a * count_b / tl.maximum(count,
1.0)
    return new_mean, new_m2, count

@triton.jit
# pyre-ignore[3]: Triton JIT kernel; no meaningful return type.
def welford_kernel(
    # pyre-ignore[2]: Pointer
    X,
    # pyre-ignore[2]: Pointer
    Mean,
    # pyre-ignore[2]: Pointer
    Var,
    N: int,
    # pyre-ignore[2]: May specialize
    stride_x,
    BLOCK_SIZE: tl.constexpr,
):
    """
    Welford one-pass reduction over the last dimension of a 2D tensor.
    Each program instance handles one row.
    """
    row_idx = tl.program_id(0)
    x_ptr = X + row_idx * stride_x

    # Initialize Welford accumulators
    mean = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
    m2 = tl.zeros([BLOCK_SIZE], dtype=tl.float32)
    count = tl.zeros([BLOCK_SIZE], dtype=tl.float32)

    # Process the row in chunks of BLOCK_SIZE
    for off in range(0, N, BLOCK_SIZE):
        cols = off + tl.arange(0, BLOCK_SIZE)
        mask = cols < N
        x = tl.load(x_ptr + cols, mask=mask, other=0.0).to(tl.float32)

```

```

# Welford update for each element in the block
new_count = count + tl.where(mask, 1.0, 0.0)
delta = x - mean
new_mean = mean + tl.where(mask, delta / tl.maximum(new_count, 1.0), 0.0)
delta2 = x - new_mean
new_m2 = m2 + tl.where(mask, delta * delta2, 0.0)

mean = new_mean
m2 = new_m2
count = new_count

# Reduce the per-lane accumulators across the BLOCK_SIZE lanes
final_mean, final_m2, final_count = tl.reduce(
    (mean, m2, count),
    axis=0,
    combine_fn=welford_combine,
)

tl.store(Mean + row_idx, final_mean)
tl.store(Var + row_idx, final_m2 / final_count)

```

This is an example of a more complex reduction. Here we are tracking 3 outputs, as even though we don't store the count and it should always be integral the mean and m2 depend on it and each output depends on multiple input tensors.

Ideally our tooling can handle analyzing this type of kernel, possibly by focusing on one output at a time.